Software Requirements Specification (SRS) Document: Template

Project Title: *Civic Innovation Through You (CITY)*

Version: *2.0*

Document Change History: *Provide a record of changes made to the document, including dates and descriptions of revision*
02/14/26 - Removing DynamoDB and updating to Sqlite3 due to built-in functionality with Django framework. Updated interface links to figma proto types.
02/03/26 - Removing Discourse and replacing it with Django forum package following a revision in strategy for providing this feature.
12/09/25 - Document Version 1.0 Final polish and submission for review
12/07/25 - Updated requirements. Added interfaces for software, hardware, and communication.
12/02/25 - Updated with requirements and user/admin interfaces
12/01/25 - Document Creation with introduction, definitions, descriptions, features, user classes, and constraints.

1. Introduction

1.1 Purpose

The purpose of this document is to define the requirements for the development of a civic innovation portal, Civic Innovation Through You (CITY). This document outlines the functional and non-functional requirements of the system to ensure a shared understanding among project collaborators, including stakeholders and developers.

CITY is a web-based platform designed to encourage residents to engage in discussions surrounding civic innovation ideas to better inform city officials in their decision making. This system provides idea submission, a reddit-style forum, polling mechanisms, and rewards system where points earn contributors badges and the chance to be featured on a leaderboard. By offering these features, the portal aims to create a fun and compelling socially based environment where people feel encouraged to share their ideas and help shape their community.

1.2 Scope

This document outlines the functional and non-functional requirements, external interfaces, system features, and other specifications necessary for the successful development of the software.

This platform will help residents and city officials engage in casual conversations for the purposes of idea generation. This platform will not facilitate legally binding agreements or official voting.

Key Features Include:

- *Idea submission forms for residents and a corresponding AI assisted inbox for city officials*
- *Reddit style forums for discussion among residents with moderation tools for city officials*

Extended Features Slotted for Later Releases Include:

- *Polling/voting mechanisms for gauging public support*
- *Gamification: with a points based system, badges, and a leaderboard*

1.3 References

[List any external documents or references used in creating this SRS.]

- *IEEE Standard 830-1998*

1.4 Definitions and Abbreviations

[Provide definitions of terms and abbreviations used throughout the document.]

- *Administrators, Admins: The city officials using the app*
- *AWS: Amazon Web Services*
- *CITY: Civic Innovation Through You*
- *FR: Functional Requirements*
- *Gamification: Users earning points to receive badges and compete for leaderboard positions to drive engagement*
- *NFR: Non-Functional Requirements*
- *Polling: Mechanism for collecting simple feedback on specific civic topics*
- *SRS: Software Requirements Specification*
- *Stakeholders: City officials, developers, and residents involved in or affected by the system*
- *Users: the residents using the app*

2. Overall Description

2.1 Product Perspective

Describe how the software fits into the larger context, including dependencies and interactions with other systems.

System Boundaries: The application will be self-contained for core logic and user management but it relies on external third party services for specific modules, allowing for rapid deployment while leveraging existing tools.

External Dependencies: This software will be web/cloud based, hosted in the AWS ecosystem. Forums will be implemented using a Django forum Package.

Client Environment: The application will interact with web browsers for its primary client environment. Resident users and city admins will interact with the app through separate interfaces.

2.2 Product Features

List the high-level features and functionalities of the software.

- *Account creation and management with authentication*
- *Submission form submission and analyzation*
- *Forums with commenting, (auto)moderation, flags, upvoting*
- *Polling creation, submission, and reporting (release date TBD)*
- *Gamification element - points system with leader board and badges reward (release date TBD)*
- *Engagement analytics (release date TBD)*

2.3 User Classes and Characteristics

Describe the intended user classes and their characteristics.

City Officials (Administrators) - Civic leaders and workers in Bellevue who review and inform policy and budgets. Their driving goal is to get input from users. They need to retain a degree of control over community discussions and have the ability to enforce community safety through moderation and account management tools. Eventually, they would benefit from having the ability to analyze metrics including poll outcomes and trends (both by topic and user engagement). Overall, the system should assist in these tasks while minimizing the time required to do so, ideally through automation and a clear, easy to use interface.

Residents (Users) - Are residents who live or work in Bellevue. Age range and population demographics may be diverse and subject to change over time. User experience needs to be fun and engaging with a low barrier to sign up. The user interface should be clean, minimalist, and easy to navigate. Account optional for viewing forum discussions to encourage curiosity and signup, but required for participation to ensure community safety.

2.4 Operating Environment

- *Client Side: Supported browsers (Chrome, Edge, Firefox, Safari) and devices (desktop, laptop, mouse, keyboard).*
- *Server Side: Web/Cloud based app using AWS ecosystem (EC2 hosting, Cognito authentication), Django framework. Scalability, backup/recovery, and security are built into the AWS ecosystem.*
- *Other: Internet connectivity, HTTPS security.*

2.5 Design and Implementation Constraints

- *Cloud based using AWS ecosystem*
- *Forums based on Django forum package*
- *Polls based on Google Forms*
- *Authentication managed through AWS Cognito*

3. Specific Requirements

3.1 Functional Requirements

List and describe the functional requirements of the software in separate sections. Use concise language and bullet points, and structure the requirements in the user story format and give each requirement a unique ID:

- *FR #1: As a new visitor, I want to view the forums and polls so I can determine if joining the app appeals to me without having to sign up first.*
- *FR #2: As a new resident user, I want to create an account so that I can submit ideas and participate in polls or discussions that catch my interest.*
- *FR #3: As a returning resident user, I want to log in so that I can submit new ideas and engage with ongoing forum discussions and active polls.*
- *FR #4: As a resident user, I want to submit a new idea with title, description, and category so that city admins can understand my ideas and concerns.*
- *FR #5: As a resident user, I want to browse discussion threads so that I can discover what others are proposing.*
- *FR #6: As a resident user, I want to filter discussion threads by category so that I can narrow down the list to topics that interest me.*
- *FR #7: As a resident user, I want to comment on ideas so that I can provide feedback, questions, or suggestions and feel like my voice matters.*
- *FR #8: As a resident user, I want to upvote ideas so that I can show support for proposals I agree with.*
- *FR #9: As a resident user, I want to report a comment as inappropriate so that the admins can review it and ensure that discussions feel safe and inclusive.*
- *FR #10: As a resident user, I want to view active polls so I can see what topics are being decided.*
- *FR #11: As a resident user, I want to vote in polls so I feel like I have a say in how my community is managed.*
- *FR #12: As a resident user, I want to see the results of polls so I can determine if my preferred results were successful.*
- *FR #13: As a resident user, I want to earn points for engagement so that my participation feels rewarding.*
- *FR #14: As a resident user, I want to see my badges so that I feel recognized for engagement.*
- *FR #15: As a resident user, I want to see a leaderboard of top contributors so that I can participate in friendly competition with others.*
- *FR #16: As an admin, I want to log into my account so that I can view and manage platform data.*
- *FR #17: As an admin, I want to create forums so that resident users can discuss their proposals.*
- *FR #18: As an admin, I want to moderate forums so I can ensure that conversations are productive and appropriate.*
- *FR #19: As an admin, I want to suspend or ban users who violate community guidelines so that the platform remains safe and constructive.*

- *FR #20: As an admin, I want to create polls so that resident users can actively vote for their input.*
- *FR #21: As an admin, I want to see how many votes an idea has so that I can gauge its community support.*
- *FR #22: As an admin, I want to view analytics so that I can track engagement trends for topics and overall platform usage.*

3.2 Non-Functional Requirements

List and describe the non-functional requirements, such as performance, security, and usability.

- *NFR #1: The performance of the platform, powered by AWS, shall support unlimited concurrent users with an average load time of under 3 seconds.*
- *NFR #2: Forum activity (commenting, upvoting) shall happen dynamically with updates posting in under 1 second.*
- *NFR #3: The system shall handle peak traffic without degradation in performance.*
- *NFR #4: The platform shall maintain 99.9% uptime availability per month.*
- *NFR #5: The security of the platform shall use HTTPS, which is enforced through AWS.*
- *NFR #6: Accounts shall be password protected and use authentication for login.*
- *NFR #7: Admin accounts shall support multi-factor authentication.*
- *NFR #8: User data shall be encrypted at rest and in transit using industry standard protocols.*
- *NFR #9: The platform shall comply with applicable data privacy regulations.*
- *NFR #10: The user interface of the platform shall be clear and simple for easy navigation and usability.*
- *NFR #11: The platform shall support current versions of Chrome, Edge, Firefox, and Safari on desktop browsers, as well as Windows, macOS, and Linux.*
- *NFR #12: The system shall recover from failures within 60 seconds using AWS auto-scaling and failover mechanisms.*
- *NFR #13: The system shall ensure transaction integrity for upvoting and downvoting (no duplicates or lost entries).*
- *NFR #14: The platform shall auto-scale horizontally to support increased user demand without downtime.*
- *NFR #15: The system shall be designed with modular features to ensure that all features may be developed independently.*
- *NFR #16: The system shall provide monitoring and logging tools to track errors, performance, and user activity.*
- *NFR #17: Poll results shall update in real time as votes are submitted.*

5. External Interface Requirements

List and describe the external interfaces, including user interfaces, hardware interfaces, software interfaces, and communication interfaces. Provide mock-ups of the user interfaces (UI) and give each interface a unique ID

Admin Interface:

Civic Innovation Through You (CITY) - Prototype1

Admin interface will provide tools for forum moderation, poll creation, analytics, and user account management. There will be a navigation bar at the top with an option to switch to user view and key analytics on the landing page.

User Interface:
Civic Innovation Through You (CITY) - Prototype1
User interface will display idea submission, forums, polls, leaderboard, and profile page with badges, along with navigation among features. There will be a navigation bar at the top

Hardware Interface: No special requirements. Desktops/laptops with keyboard, mouse, and internet connection to navigate our web app.

Software Interface:

- AWS Services (for hosting, authentication, cloud infrastructure)
- Django forum package Integration (for forum functionality and moderation tools)
- Google Forms Integration (for polls)
- Browser environment (latest versions of Chrome, Edge, Firefox, and Safari running on Windows, macOS, and Linux)

Communication Interface:

- HTTPS Protocol (encrypted between client and server)
- RESTful APIs (exposed/consumed for between App and Google Forms)
- Authentication (AWS Cognito)
- WebSockets (dynamic updates to forums and polls)

6. Constraints
[List any constraints or limitations that might impact the software development.]

- *Budget = minimal. Limited funding restricts which third party services may be used.*
- *Time = by Spring Quarter End. Limits feature development to a tight work scope.*
- *Dev Team Stability = losing team members necessitates changes to scope.*
- *Security = anti-harassment/spam/flooding concerns around public forum/submissions*
- *Regular administrative work required, necessitating staff commitment*
- *AWS ecosystem limits options to this environment and reduces flexibility for later migration*

6. Assumptions and Dependencies
[Outline any assumptions made during the requirement gathering process and dependencies on external systems or components.]

- Django forum packages will be robust enough to support our intended user experience
- Sqlite3 can store our user and post data effectively.
- AWSCognito can effectively store user data for authentication purposes.
- We can use Django and Django templates to build our front end to an acceptable degree of quality.

7. Appendices

[Include any additional information, such as a glossary, use case diagrams, or entity-relationship diagrams.]